

ContractDB: Secure and Efficient Integration of Large Legacy Data With Blockchain DApps

Yingjie Xue , *Member, IEEE*, Xinyi Luo, Meiqi Li, Kaiping Xue , *Senior Member, IEEE*, Yunshu Wang ,
and Lutong Chen , *Member, IEEE*

Abstract—The rapid growth of blockchain-based decentralized applications (DApps) highlights blockchain’s potential to enhance application security. However, expensive on-chain data storage limits the deployment of DApps with large datasets. Additionally, DApp development tools, such as Ethereum smart contracts, lack support for complex queries, further hindering data-intensive DApps. To address the challenges of expensive storage and inability of complex queries, we propose ContractDB, a framework that integrates external verifiable databases (VDBs) with blockchain DApps for verifiable query processing. ContractDB offloads data storage and processing to VDBs, thereby reducing on-chain storage costs and enhancing data handling capabilities. Existing VDBs incur high verification costs and lack support for public verifiable update. In this paper, we propose a novel VDB design using authenticated dictionaries and authenticated set operations to reduce verification cost and enable verifiable updates. We further introduce a disputable update mechanism that leverages public verification to lower update costs. Performance evaluations show that ContractDB can verify the results of 6-condition conjunction (with mixed equivalent and range) queries on a 2^{20} -line data table within 2.4 million gas cost, with potential for optimization. For comparison, storing those data in contracts requires over 36 billion gas and still cannot support range or multi-condition queries. Therefore, the proposed ContractDB makes it feasible to support DApps with large datasets.

Index Terms—Blockchain, verifiable database, smart contract, DApp.

I. INTRODUCTION

A DECENTRALIZED application (DApp) refers to an application without the need for a centralized server or authority to manage data and execute codes [1]. DApp technology has garnered considerable attention because of its ability to enhance security and universality by eliminating reliance on centralized servers, thereby enabling flexible interaction among participants. The core principle underlying DApps is consistency, wherein all participants share the same view of data and

execution flows. Consequently, DApps predominantly operate on blockchains in the form of smart contracts. Blockchain, well-known due to the emergence of Bitcoin [2], establishes a reliable decentralized ledger that is tamper-resistant, allowing network participants to collaboratively record data on the ledger while ensuring consistency through decentralized consensus. Subsequently, Ethereum [1] expands the blockchain from a mere ledger to a decentralized system that supports Turing-complete programming languages, known as smart contracts, triggering the outburst of DApp development. DApp developers can implement their applications via smart contracts, which, once deployed on-chain, execute deterministically according to their programmed logic.

Motivation: Despite the advantages of smart contracts, their limitations in data storage and processing capabilities hinder the development of blockchain DApps. First, on-chain storage is expensive. For instance, storing 32-byte data in an Ethereum contract requires 20,000 gas, nearly equivalent to the cost of a basic transaction [3]. Second, smart contracts only support key-value queries, complicating the execution of complex queries. This limitation stems from the authenticated data structures used in the blockchain, such as the Merkle Patricia trie or vector commitment. Given these limitations, blockchain DApps struggle with handling large-scale data applications. Although various efforts have been made to improve blockchain performance, such as consensus optimization [4], [5], sharding [6], [7], off-chain payment channels [8], [9], and concurrent execution [10], [11], these solutions focus on improving the throughput of transactions, rather than overcoming limitations of data storage and data query.

Currently, there are several technical methods to enhance the data storage and processing capabilities of blockchain. Regarding data storage, existing solutions offload data and related transactions to off-chain nodes, recording only short commitments on-chain. However, generating commitments often relies on several trusted nodes [12] or trusted execution environments [13], [14], adding the reliance on additional secure components. In terms of data processing, there are several works which aim to enable complex queries of on-chain data [15], [16], [17]. They utilize authenticated data structures (ADS) like Merkle B-trees to build indexes for on-chain data. Integrating them into DApp contract poses non-trivial challenges in terms of verification, as the on-chain verification incurs significant overhead.

Our Work: ContractDB aims to enhance the data storage and processing capabilities of smart contracts, enabling blockchain

Received 4 January 2026; revised 14 May 2026; accepted 2 July 2026. Date of publication 8 July 2026; date of current version 1 September 2026. This work was supported in part by the National Natural Science Foundation of China under Grant U25A20427, Grant 62372425, and Grant 62501562, in part by Anhui Province Key Technologies Research & Development Program under Grant 2022a05020050, and in part by the Youth Innovation Promotion Association of the Chinese Academy of Sciences under Grant Y202093. (Corresponding author: Kaiping Xue.)

Yingjie Xue, Xinyi Luo, Meiqi Li, Kaiping Xue, and Yunshu Wang are with the School of Cyber Science and Technology, University of Science and Technology of China, Hefei 230027, China (e-mail: kpxue@ustc.edu.cn).

Lutong Chen is with the Network and Information Center, University of Science and Technology of China, Hefei 230026, China.

Digital Object Identifier 10.1109/TDSC.2026.3711164

DApps to integrate and utilize large datasets. Our main idea is to leverage external verifiable databases (VDBs) for data storage and access. A VDB uses authenticated data structures to generate a digest that allows users to verify query results without accessing the full database. The term “external” indicates that the VDB is hosted on one or several servers, not replicated across all blockchain nodes. A simple example can be found in Section II-A.

While the proposed approach seems straightforward, existing VDB schemes face significant challenges when integrated with smart contracts. One key challenge is the public verification of query results. Once a query result is returned, “the blockchain” needs to verify its correctness. Currently, there are two kinds of VDB constructions, including circuit-based [18], [19] and accumulator-based [20], [21], [22] constructions. Circuit-based constructions are inefficient for public verification. As [19] shows, it may take several minutes to generate a query proof based on zero-knowledge proofs. Thus, accumulator-based solutions are more suitable. However, existing accumulator-based solutions still have limitations. In brief, the main challenges include large verification costs and lack of public verification mechanisms when database is updated, with a detailed discussion available in Section II-C.

To make VDB usable in DApps, we focus on solving two remaining problems in accumulator-based solutions. We propose a new VDB construction that reduces verification costs compared to existing constructions and enables public verification for updates. For query authentication, our scheme combines authenticated dictionaries and verifiable set operations. Compared to existing tree-based schemes, our approach reduces the verification complexity from $O(k \cdot \log N)$ to $O(k)$, where N and k denote the total number of data items and the number of query conditions, respectively. This improvement primarily stems from the use of constant-size membership proofs in our authenticated dictionary construction.¹ Specifically, we propose a solution to reduce verification cost of multi-condition queries. That solution is presented in our previous work.²

In this paper, we further optimize the cost of update verification. We introduce an efficient update verification mechanism to further lower the cost of update verification. Our intuition is that, while proving the correctness of an updated digest is difficult, proving the incorrectness is simple, as long as the contract knows the correct query result. We design a disputable update mechanism based on this consideration which mirrors the core idea of optimistic rollup [23]. To conclude, our main contributions are as follows:

- We propose ContractDB, which leverages verifiable databases to enhance the data storage and processing capabilities of smart contracts. Our work fills the gap in handling large data within blockchain DApps.

¹Tree-based authenticated indexes require a root-to-leaf proof for each condition, leading to an $O(\log N)$ cost per condition and $O(k \cdot \log N)$ overall. In contrast, our approach achieves $O(1)$ verification complexity per condition. The total verification complexity for k conditions is $O(k)$.

²The previous work “ContractDB: Enabling Secure and Efficient DApps via Integrating Blockchain and External VDBs” appears in the proceedings of the 2025 International Conference on Parallel and Distributed Systems.

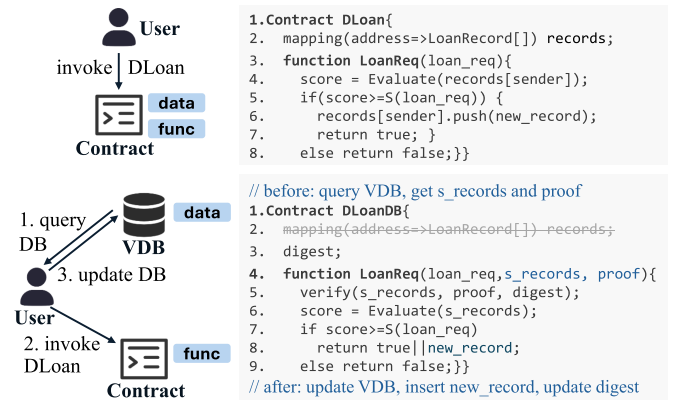


Fig. 1. The DLoan Contract Assisted By VDB.

- To enable public verification of VDB, we design a scheme that combines authenticated dictionaries and authenticated set operations. In addition, we design a tier index to reduce the verification cost of multi-condition queries. Comparing to existing schemes, the verification costs in our scheme are reduced from $O(k \log N)$ to $O(k)$, where N and k represent the total quantity of data items and the number of query conditions, respectively.
- To reduce the update costs of VDB, we propose a disputable update mechanism. Our basic idea is to leverage public verification to lower the contract verification costs. A dispute mechanism is provided to ensure the security of public verification.
- We implement a prototype of ContractDB and conducted extensive experiments to evaluate its performance. The results show that it takes around 2.4 million gas to verify the query result on a 6-condition conjunction sum query for a 2^{20} -line data table. In comparison, merely storing the data table in contracts requires over 36 billion gas. ContractDB offers a new opportunity for blockchain contracts to support DApps with large-scale data.

The rest of the paper is organized as follows. Section II presents a motivation example and related work. Section III introduces background knowledge, including accumulators, authenticated set operations, and authenticated dictionaries. Section IV provides the system and threat model, design goals, and the outline of ContractDB. Sections V and VI present our VDB construction and the disputable update mechanism. Sections VII and VIII offer security and performance analyses. Finally, Section IX concludes our work.

II. MOTIVATION AND RELATED WORK

A. Motivation Example

In many cases, a DApp depends on large datasets with complex filter conditions. For instance, suppose that multiple banks decide to initiate a decentralized loan application on Ethereum (the DLoan contract, as shown in Fig. 1), with each bank possessing a large-scale dataset containing historical loan records. When users invoke the Loan function to submit a loan request, the Evaluation function is executed to assess the user and determine

whether to approve the request. The Evaluation function takes the relevant loan records queried from the dataset as input. When implemented through smart contracts, such a DApp faces two challenges.

- **Expensive on-chain storage.** All banks must upload their dataset to the contract so that functions can access the required data. However, on-chain storage for a large dataset can be extremely costly. In Ethereum, it costs 20,000 gas to store 32 bytes in a contract, and the contract size is limited to 24 KB. For our experimental dataset (Section VIII-A), it requires more than 36 billion gas and 2,389 contracts to store a 2^{20} -line data table.
- **Inflexible data query.** The Evaluation function may require a user's loan records with a specific query condition, such as `SELECT SUM(amount) FROM records WHERE user='User1' AND amount BETWEEN 10 AND 20 AND date > '2024-10-01'`. However, a contract supports only a key-value index based on mapping, making it inconvenient or expensive to process such queries.

We propose to leverage external databases to assist contracts. As depicted in Fig. 1, the DLoanDB contract is supported by an external verifiable database, named VDB. Datasets are stored in VDB whose digest is recorded in DLoanDB. When users invoke the LoanReq function, they should first query the VDB to obtain the query results and proofs, which they then add to the invocation input. The LoanReq function can then verify the query results using the digest and proofs. Subsequently, if LoanReq approves the request, it may generate new records to be inserted into VDB. After VDB processes the insertion, it updates the digest stored in DLoanDB. In essence, we trade communication costs for storage costs and query capabilities. This tradeoff can be particularly advantageous when the dataset is large.

B. Existing Blockchain & Database Systems

The combination of blockchain and databases has been widely studied in recent years [24], [25]. From the perspective of the system model and goals, this combination can be classified into three types. Based on their functions, we refer to them as blockchain-protected databases, blockchain-based databases, and database-assisted blockchains.

Blockchain-assisted databases [26], [27], [28] aim to use blockchains to enhance the security of outsourced or shared databases. The main idea is to record the database digest and logs on-chain, allowing for user verification and mutual audits. An important requirement is to enable the blockchain to determine the correctness of the digest, especially after database updates. To this end, FalconDB [26] requires each blockchain node to maintain a copy of the database so that they can verify the digest. In contrast, [28] and [27] design gas-efficient digest constructions that allow a smart contract to efficiently verify the update of a digest. However, their query capabilities are limited to single-condition queries without result aggregation (e.g., summation or average).

Blockchain-based databases [17], [29], [30] aim to design a secure decentralized database for data on blockchains. The key

point is to treat each blockchain transaction as a data object and create authenticated indexes on them, enabling complex queries on data objects stored on-chain. An important challenge is to flexibly handle query demands at different levels with appropriate indexes, such as skipping blocks that do not contain the required data to reduce query costs. To this end, SEBDB [29] designs a layered index that leverages B^+ -trees and bitmaps to enable SQL queries on on-chain data. vChain+ [17] utilizes the bilinear accumulator as their underlying technology. These solutions design authenticated indexes for the blockchain-form datasets. However, they do not support public verifiable update.

Database-assisted blockchains leverage databases to optimize the data storage and processing capabilities of blockchains, and our solution falls within this area. Some solutions utilize off-chain storage systems such as IPFS to reduce on-chain storage costs, and the data is not used by contracts. However, we focus on situations where data stored in databases will be used during contract execution. A typical example is Hyperledger Fabric [31], which enables blockchain nodes to use CouchDB as the ledger and interact with CouchDB in the chaincode (i.e., contract). While database-assisted blockchains like Hyperledger Fabric integrate systems such as CouchDB to support rich queries, they typically operate under a permissioned trust model. Query results are verified by the network's consensus rather than through public, cryptographic proof. Consequently, such systems lack public verifiability; entities outside the permissioned set cannot independently validate query or update results without trusting the underlying infrastructure.

C. Existing Verifiable Databases

A verifiable database (VDB) scheme enables a client to check the correctness of outsourced database query without accessing the origin dataset, usually SQL queries containing equivalent query, range query, etc. Typically, the database owner uses authenticated data structures (ADS) such as Merkle B-Tree or accumulators to generate a digest of the database. Then, the owner outsources the database and ADS to a server and keeps the digest. Subsequently, the owner or other requesters can send SQL queries to the server. The server generates a proof for the query result based on which the requester can check the correctness.

Earlier VDB constructions [32] leverage Merkle B-Tree to enable verifiable range queries. To enable more flexible query such as multi-condition queries and aggregating (e.g., summation, average, max-min) queries, there are two main technical methods, including accumulator-based solutions and circuit-based solutions.

Circuit-based solutions [18], [19] consist of interactive and non-interactive constructions. Since the interactive construction does not support public verification, only snark-based non-interactive constructions have the potential to be used in our work. However, as evaluated in [19], it may take several minutes to generate a proof. Such a cost is impractical.

Accumulator-based solutions [20], [21], [22] leverage authenticated set operations (ASO) based on bilinear pairing or RSA accumulators to enable verifiable queries on complex conditions.

These solutions are designed for outsourced databases and rely on a trusted data owner to handle data update. Thus, they do not support public verifiable updates. However, in our scheme, the contract should be able to verify whether a updated digest is correct without accessing the original database. The update demands may have query conditions such as SQL update, deletion, or insertion queries. Currently, existing accumulator-based solutions cannot support public verification on the update queries.

III. PRELIMINARIES

This section introduces three key technologies used to construct the ContractDB protocol. First, we introduce the bilinear pairing-based accumulator proposed in [22], which enables verifiable subset checking and set summation. Second, we describe the authenticated set operation scheme proposed in [33], [34], which facilitates verifiable set intersection and union. Last, we present the authenticated dictionary scheme proposed in [35], which allows for verifiable key-value queries.

A. Accumulator

An accumulator compresses a set into a succinct digest. With an accumulator, one can verify whether one or multiple data items are in the set according to the digest, without accessing the original set. There are several constructions for accumulators, and we adopt the construction based on bilinear pairing and characteristic polynomial [22], [34], which supports summation queries of a set.

- *Bilinear Pairing (on elliptic curves)*: Let $\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T$ be cyclic multiplicative groups of prime order p . Let $e: \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$ be a bilinear pairing satisfying (1) $e(aG_1, bG_2) = e(G_1, G_2)^{ab}$ for all $G_1, G_2 \in \mathbb{G}_1 \times \mathbb{G}_2$ and $a, b \in \mathbb{Z}_p$, (2) $e(G_1, G_2) \neq 1$ where G_1, G_2 are generators of $\mathbb{G}_1$ and $\mathbb{G}_2$, respectively, and (3) $e(G_1, G_2)$ is easy to compute. We denote the bilinear pairing parameters as $\text{pub} = (p, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, G_1, G_2, e)$.
- *Characteristic Polynomial*: Given a set $S = \{d_1, \dots, d_n\}$, $d_i \in \mathbb{Z}_p$, the characteristic polynomial of S is defined as $P_S(x) = \prod_{i=1}^n (x + d_i^{-1}) = \sum_{i=0}^n a_i x^i$. A $P(x)$ uniquely represents a set.

An accumulator protocol supporting summation consists of the following functions. Note that some functions require a portion of pk as input, which we omit for brevity. The required portion of pk of each function can be found from its algorithm.

- 1) $\text{pk} \leftarrow \text{GenKey}(\text{pub}, s)$. On inputting pub and a secret key $s \in \mathbb{Z}_p$, it computes $\text{pk} = (G_i, sG_i, \dots, s^N G_i)$ for $i \in \{1, 2\}$, where N is the maximum length of a set.
- 2) $f_S / F_S \leftarrow \text{Acc}(S, G_i)$ ($i \in \{1, 2\}$). On inputting a set $S = \{d_1, \dots, d_n\} \subset \mathbb{Z}_p$ where $n \leq N$, it outputs the accumulator of S , denoted by f_S ($i=1$) or F_S ($i=2$). Specifically, it computes $P_S(x) = \sum_{j=0}^n a_j x^j$ and $f_S / F_S = \sum_{j=0}^n a_j s^j G_i$, where $s^j G_i$ is in pk .
- 3) $w_{\text{subset}} \leftarrow \text{ProveSub}(S_1, S_2)$. On inputting two sets S_1 and S_2 , if $S_1 \subseteq S_2$, it computes $D = S_2 \setminus S_1$ and $P_D(x) = \sum_{i=0}^n a_i x^i$. Then, the subset witness is given by $w_{\text{subset}} = \sum_{i=0}^n a_i s^i G_2$. S_1 can be a single element or a set.

- 4) $0/1 \leftarrow \text{IsSub}(f_{S_1}, f_{S_2}, w_{\text{subset}})$. On inputting two set accumulators f_{S_1} and f_{S_2} along with a subset witness w_{subset} , it outputs 1 if $e(f_{S_1}, w_{\text{subset}}) = e(f_{S_2}, G_2)$, indicating that $S_1 \subseteq S_2$. Otherwise, it returns 0.
- 5) $w_{\text{sum}} \leftarrow \text{ProveSum}(S)$. On inputting a set S , it outputs a sum witness w_{sum} . Assume $P_S(x) = \sum_{i=0}^n a_i x^i$, then $w_{\text{sum}} = (a_0, a_1, w_1, w_2)$, where $w_1 = \sum_{i=0}^{n-1} a_{i+1} s^i G_1$ and $w_2 = \sum_{i=0}^{n-2} a_{i+2} s^i G_1$.
- 6) $0/1 \leftarrow \text{IsSum}(\text{sum}, f_S, w_{\text{sum}})$. On inputting an integer sum, a set accumulator f_S , and a sum witness w_{sum} , it outputs 1 if (1) $\text{sum} = a_1 a_0^{-1} \bmod p$, (2) $e(w_1, sG_2) = e(f_S - a_0 G_1, G_2)$, and (3) $e(w_2, sG_2) = e(w_1 - a_1 G_1, G_2)$.

The security of sum and IsSum is proven in [22], and the security of the other protocols is established in [34]. To be noted, the secret key s is utilized exclusively during the one-time setup algorithm $\text{GenKey}(\text{pub}, s)$ to derive the public parameters pk . Following this initialization, all subsequent index construction, proof generation, and verification procedures rely solely on pk . Consequently, no party is required to retain s , and it can be securely erased to prevent the forgery of subset proofs.

Additionally, although pk is large, the verification functions require only the first several elements of pk . Therefore, our scheme does not bring additional storage overhead to verifiers.

B. Authenticated Set Operation

An authenticated set operation (ASO) [34] enables a client to verify the correctness of outsourced set operations, such as intersection and union, performed by an untrusted server. For instance, given a group of sets denoted by $S_1, \dots, S_k$, the server computes $I = \bigcap_{i=1}^k S_i$ and returns I to the client. With ASO, the client can then confirm the correctness of I without knowing the individual sets S_i . ASO is a fundamental component in constructing a verifiable database. Omitting auxiliary information such as public parameters, an ASO scheme consists of the following algorithms.

- 1) $(\Pi_{\text{op}}) \leftarrow \text{ProveOp}(S_1, \dots, S_k, \text{op}, R)$. On inputting a group of sets $(S_1, \dots, S_k)$, an operation instruction op , and the operation result R , it generates a proof Π_{op} used to verify R . Additionally, op can represent a single intersection or union, or hierarchical set operations.
- 2) $0/1 \leftarrow \text{IsOp}(f_1, \dots, f_k, f_R, \text{op}, \Pi_{\text{op}})$. On inputting a group of accumulators $f_1, \dots, f_k, f_R$, op , and Π_{op} , it verifies whether f_R is the accumulator of a set R which is the correct result of op , returning 1 if correct; otherwise, it returns 0.

C. Authenticated Dictionary

An authenticated dictionary (AD) enables a client to verify the results of outsourced key-value searches. We leverage the AD scheme proposed in Aardvark [35]. In brief, suppose there is a set of key-value data items (k_i, v_i) ($i \in [1, n]$) sorted by k_i . Each data item contains a reference to the next key, i.e., $\text{item}_i = (k_i, v_i, \text{next}(k_i) = k_{i+1})$. Assume $D = \{\text{item}_i\}$ and compute $\delta_D = \text{Acc}(D)$; δ_D serves as the digest of the dictionary.

Subsequently, one can query a certain key k from the server that stores D to verify the correctness of the query result with the help of δ_D . The principle is that if k exists in the dictionary, there must be an item i such that $k_i = k$; and if k does not exist, there must be an item i such that $k_i < k < k_{i+1}$. Thus, the server simply returns the corresponding item i and its subset witness generated by $\text{ProveSub}(d_i, D)$.

IV. CONTRACTDB OVERVIEW

A. System and Threat Model

The ContractDB system consists of a blockchain, one or several database servers, and some users. Our scheme can be used in any blockchain that supports smart contracts and we use Ethereum in our implementation. Database servers maintain databases and do not participate in the blockchain consensus. Users are clients who need to deploy or invoke smart contracts which are assisted by external databases.

We assume the blockchain is secure. Thus, contract invocations will be executed according to the code. Users and database servers can be malicious. Users may attempt to submit incorrect database query results during contract invocation. We assume an adversarial model where all database servers may be malicious. The correctness property—ensuring that the contract rejects any incorrect query result—must hold even under this worst-case assumption. Besides, for a client to successfully retrieve a query result—a property we define as data availability—the system requires the existence of at least one responsive server that provides the necessary data. We assume that there is at least one such server in the network. In summary, our threat model assumes that all servers may be malicious; the only operational requirement is the existence of at least one responsive server.

In addition, when a database is initially integrated to the blockchain, an initial digest is uploaded to the contract. We assume that the initial digest is correct.

B. Design Goals

Our primary goal is to enable smart contracts to offload data to external databases, thereby enhancing data storage and processing capabilities. Such a scheme requires two security goals and several performance optimizations.

1) *Correctness*: Correctness requires that the contract can verify the database query and update results. This requirement can be easily transferred to the correctness of the adopted VDB scheme. The correctness of the VDB query requires that given a database D , a digest δ , a query instruction q , and a result R , the query verification only outputs true when R is the result of querying q on D and δ is the digest of D . The correctness of the VDB update requires that given a database D , an update instruction u , and two digests δ and δ' , the update verification only outputs true when δ is the digest of D and δ' is the digest of updating D using u . It is important to distinguish between data availability and data correctness within our design goals. Availability requires at least one database server to be responsive to process queries; however, we do not assume the honesty of such a server, as it may return incorrect results. While

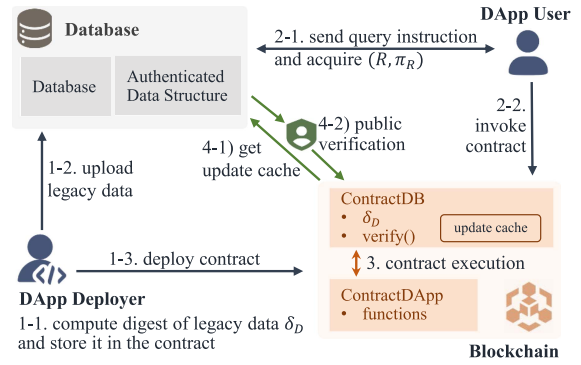


Fig. 2. ContractDB Outline.

availability may be compromised in the extreme case where all database servers are offline or unresponsive, a core design goal of ContractDB is to guarantee correctness. This ensures that the blockchain never accepts incorrect data or invalid digests, even if all servers act maliciously.

2) *Consistency*: Consistency requires that the blockchain and VDB server handles database instructions in the same sequence. Assume the blockchain handles a set of contract invocation transactions $(T_1, \dots, T_k)$ sequentially, where each T_i contains two database instruction (q_i, u_i) with a query-then-update sequence. Then, consistency persists if the database handles instructions in the order of $(q_1, u_1, q_2, \dots, u_k)$. Consistency requires that once the contract generates an update demand, any query results generated before VDB executing the update become incorrect.

3) *Efficiency and Compatibility*: To ensure correctness and consistency, we adopt the VDB scheme, which introduces additional storage and communication overhead. Therefore, our first efficiency goal is to reduce the verification cost of VDB queries and updates. In addition, we require that the proposed ContractDB scheme does not modify blockchain protocols, ensuring compatibility with existing blockchains.

C. ContractDB Outline

In brief, ContractDB leverages external verifiable databases (VDB) to maintain large datasets. For clarity, we use the example provided in Section II-A to illustrate how ContractDB works. As shown in Fig. 2, the workflow is as follows.

1) *Contract Deploy*. As discussed in Section II-A, multiple banks decide to initiate a decentralized loan application (the DLoan contract) on the blockchain and integrate related information into a large legacy dataset that will be accessed by the DLoan contract. They record the legacy dataset into a verifiable database (VDB), generating a digest δ_D and recording it in the DLoan contract during contract deployment. Specifically, the contract deployer executes the one-time setup to generate the public parameters pk from a secret key s randomly chosen by itself, utilizes pk to construct the authenticated indexes and the initial digest, and subsequently erases s securely. This contract deployment phase serves as the trusted setup for the system.

- 2) *Contract Invocation.* A user who needs to invoke LoanReq first queries the VDB to obtain (R, π) . The query instruction is derived by pre-executing the invocation locally. Then, the user invokes LoanReq, submitting (R, π) along with other input.
- 3) *Contract Execution.* During execution, the contract verifies (R, π) based on the query instruction and the DB digest δ_D . If LoanReq generates a new loan record, it should be inserted into the loan table. To reduce update costs, the update requests are cached in the DLoan contract and periodically update the VDB. Therefore, after verifying R , LoanReq should dynamically update R based on the update cache.
- 4) *VDB Update.* When the VDB finds that the contract has run out of the update cache, it should apply the update to generate a new digest and upload it to the contract. Then, the contract verifies the new digest and releases the cache if it is correct.

Insight of Security: The ContractDB protocol ensures both correctness and consistency. Correctness is guaranteed by the verifiable property of the VDB. As long as the contract stores a correct VDB digest δ_D , it can verify the correctness of query results uploaded by users. In addition, the disputable update mechanism ensures the correctness of δ_D . Consistency is guaranteed by the caching and update mechanism. Assume that the contract stores a digest δ_D and some update demands $(u_1, \dots, u_k)$. From the view of blockchain, the current database version should be D with updates $(u_1, \dots, u_k)$. The update cache mechanism prevents the VDB from controlling the order of updates; instead, the VDB should generate results based on D and the updates are executed by the contract, thereby achieving consistency.

Challenges: According to the ContractDB workflow, the contract should verify both query and update results. Thus, the VDB should support efficient public verification for both queries and updates, which existing VDB schemes cannot adequately support, as analyzed in Section II-C. Therefore, we propose a new VDB construction that supports efficient query verification in Section V, along with an efficient dispute update mechanism in Section VI.

V. VDB WITH SUCCINCT PROOF

We combine an authenticated dictionary (AD) and authenticated set operation (ASO) to construct a VDB scheme that supports multi-dimensional aggregating queries with succinct proof and efficient verification. Intuitively, an AD scheme enables key-value queries with constant proof size, while an ASO scheme facilitates efficient multi-dimensional queries by conducting intersection or union operations on the results of each dimension.

In this section, we first define the index in Section V-A. Then, we construct the VDB scheme by three steps. First, we construct the single-cond single-dest index (Section V-B). Second, we show the index combination protocols that supports multi-condition multi-destination query by combining single-cond single-dest indexes (Section V-C). Third, we propose the

TABLE I
DATA TABLE T

Key ID	Name	Bank	Rate	Amt
d_1	Alice	Bank1	4.3	85
d_2	Bob	Bank1	4.7	90
d_3	Alice	Bank2	5.1	70
d_4	Carol	Bank3	4.9	95
d_5	Bob	Bank2	4.8	60

Amt: Amount, Rate: Interest Rate.

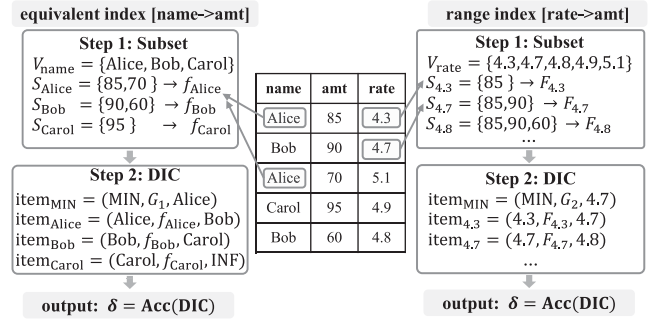


Fig. 3. Single-cond single-dest Indexes on Table I.

optimized index combination mechanism that is more efficient to support hybrid queries (Section V-D). To make the protocol more understandable, we take the example in Table I to introduce our construction.

A. Index Definition

An index consists of some data structures generated based on AD and ASO to support verifiable queries and updates on specific conditions. In a SQL select query, for example, `SELECT SUM(amt), AVG(rate) FROM T WHERE name=Alice AND amt BETWEEN 50 AND 250`, there is a destination list (dest_list) [SUM(amt), AVG(rate)], a table name T , and a condition list (cond_list) [name, amt]. We call these a select triple, and use a select triple to describe the query type that an index supported. Thus, we use $[\text{cond_list} \rightarrow \text{dest_list}](T)$ to denote an authenticated index on T and omit T when it is apparent.

B. Single-Cond Single-Dest Index

A single-cond single-dest index is an authenticated index whose cond_list and dest_list contain only one column each. In terms of conditions, there are two most common query demands: equivalent queries and range queries. We separately construct the equivalent index and range index. A column that requires only an equivalent query should create an equivalent index, as it is more cost-effective, while a column that requires both equivalent and range queries can create a range index. For the destination, we take the sum query as an example to detail our construction and extend it to other query types in Section V-B3. Fig. 3 shows a simple example of constructing equivalent and range indexes.

1) *Equivalent Index:* Take $[\text{name} \rightarrow \text{amt}]$ as an example; its construction consists of three steps. First, let V_{name} consist of all existing values in the name column in ascending order. For instance, suppose $\text{MIN} < \text{Alice} < \text{Bob} < \text{Carol} < \text{INF}$, where

MIN and INF denote infinitesimal and infinity, respectively, and $V_{\text{name}} = \{\text{Alice}, \text{Bob}, \text{Carol}\}$. Such a deterministic sort can be achieved through a hash function. Second, for each $n \in V_{\text{name}}$, generate a subset $S_n = \{d_i.\text{amt}\}$ for $d_i.\text{name} = n$, and compute $f_n = \text{Acc}(S_n, G_1)$. Finally, construct an AD on the (n, f_n) pairs. Specifically, let $\text{item}_n = (n, f_n, \text{next}(n))$, where $\text{next}(n)$ denotes the next value of n in V_{name} . The value of item_{MIN} is set to G_1 which indicates an empty subset. Then, construct a dictionary $D = \{\text{item}_n\}_{n \in V_{\text{name}} \cap \{\text{MIN}\}}$, and compute $\delta_D \leftarrow \text{Acc}(D, G_1)$. δ_D is the digest of the index $[\text{name} \rightarrow \text{amt}]$.

Storing δ_D , a contract can efficiently verify the query result on the $[\text{name} \rightarrow \text{amt}]$ index. For a query $\text{SELECT SUM}(\text{amt}) \text{ FROM TABLE T WHERE name} = n$, the query and verification process are as follows.

- $(R, \pi) \leftarrow \text{QueryEq}(n)$. The result R is $\text{sum}(S_n)$. To generate the query proof π , find an item such that $\text{item.key} = n$ and $\pi = (\text{item}_n, w_{\text{sub}}, w_{\text{sum}}, f_R)$, where $w_{\text{sub}} = \text{ProveSub}(\text{item}, D)$, $w_{\text{sum}} = \text{ProveSum}(S_n)$, and $f_R = \text{Acc}(S_R, G_1)$.
- $0/1 \leftarrow \text{VerifyEq}(R, \pi, n, \delta_D)$. Check: 1) $\text{item.key} = n$, 2) $\text{IsSub}(\text{item}, \delta_D)$ outputs 1, 3) $f_R = \text{item.val}$, and 4) $\text{IsSum}(R, f_R, w_{\text{sum}})$ outputs 1. If all checks pass, output 1; otherwise, output 0.

When the result is empty, there must be an item such that $\text{item.key} < n < \text{item.next}$. Then, the proof π is $(\text{item}, w_{\text{sub}})$. To verify an empty result, simply check whether $\text{item.key} < n < \text{item.next}$ and if $\text{IsSub}(\text{item}, \delta_D)$ outputs 1.

2) *Range Index*: Take $[\text{rate} \rightarrow \text{amt}]$ as an example; it is impractical to use an equivalent index for a range query since a range query may consist of multiple keys, leading to huge verification costs. For instance, to query “rate BETWEEN 4 AND 5”, the proof should contain three dictionary items, three subset witnesses, and three sum witnesses, as each data item has a unique rate value. Thus, to enhance the efficiency of the range index, we combine the difference set-based solution proposed in [21] with our construction. In addition, float and date values should be converted to $\mathbb{Z}_p$ values.

To create a range index of $[\text{rate} \rightarrow \text{amt}]$ on Table I, similar to the equivalent index, let $V_{\text{rate}} = \{4.3, 4.7, 4.8, 4.9, 5.1\}$. Then, for each $r \in V_{\text{rate}}$, generate a subset $S_r = \{d_i.\text{amt}\} (d_i.\text{rate} \leq r)$, and compute $F_r = \text{Acc}(S_r, G_2)$. Finally, create an AD for the (r, F_r) pairs, i.e., $\text{item}_r = (r, F_r, \text{next}(r))$, $D = \{\text{item}_r\}_{r \in V_{\text{rate}}}$ and $\delta_D = \text{Acc}(D, G_1)$.

Take $\text{SELECT SUM}(\text{amt}) \text{ FROM TABLE T WHERE range BETWEEN } v_l \text{ and } v_r$ as an example, the query and verification process of a range index is as follows.

- $(R, \pi) \leftarrow \text{QueryRange}(v_l, v_r)$. Find item_1 and item_2 satisfying $\text{item}_1.\text{key} < v_l \leq \text{item}_1.\text{next}$ and $\text{item}_2.\text{key} \leq v_r < \text{item}_2.\text{next}$. Let S_1 and S_2 be the subsets related to item_1 and item_2 , respectively, and let $S_R = S_2 \setminus S_1$. Then, the query result R is $\text{sum}(S_R)$ and the proof π is $(\{\text{item}_i, w_{\text{sub}_i}\}_{i \in \{1,2\}}, w_{\text{sum}}, f_R)$, where $w_{\text{sub}_i} = \text{ProveSub}(\text{item}_i, D)$, $w_{\text{sum}} = \text{ProveSum}(D)$, and $f_R = \text{Acc}(S_R, G_1)$.
- $0/1 \leftarrow \text{VerifyRange}(R, \pi, v_l, v_r, \delta_D)$. Check 1) $\text{item}_1.\text{key} < v_l \leq \text{item}_1.\text{next}$ and $\text{item}_2.\text{key} \leq v_r < \text{item}_2.\text{next}$; 2) both $\text{IsSub}(\text{item}_1, \delta_D)$ and $\text{IsSub}(\text{item}_2, \delta_D)$ output 1;

TABLE II
ITEM KEY CHECK

Border Type	Item Key Check
close left	$\text{item}_1.\text{key} < v_l \leq \text{item}_1.\text{next}$
open left	$\text{item}_1.\text{key} \leq v_l < \text{item}_1.\text{next}$
close right	$\text{item}_2.\text{key} \leq v_r < \text{item}_2.\text{next}$
open right	$\text{item}_2.\text{key} < v_r \leq \text{item}_2.\text{next}$

3) $e(f_R, \text{item}_1.\text{val}) = e(G_1, \text{item}_2.\text{val})$, indicating that $S_R, S_1 \subseteq S_2$ and $S_R = S_2 \setminus S_1$; and 4) $\text{IsSum}(R, f_R, w_{\text{sum}})$ outputs 1.

When encounter an empty result, item_1 and item_2 will be the same (i.e., $\text{item}_1.\text{key} = \text{item}_2.\text{key}$), and the item key check for left and right borders remains unchanged.

The between-and query is a closed-left and closed-right query, i.e., $v \in [v_l, v_r]$. For open border queries, the check of item keys is slightly different, while other processes remain the same. Table II shows the item key check for different border types. It is noteworthy that the item checks for left and right borders are independent.

3) *Other Queries*: Our VDB construction supports other queries including maximum, minimum, count, and average, consistent with IntegriDB [22]. The maximum and minimum queries are transformed into range queries. For example, if the query $\text{SELECT amt FROM T WHERE amt} \geq a_1$ returns a_1 , it indicates that a_1 is the maximum value. To enable the count query, IntegriDB proposes generating a new column (denoted by new) such that $d_i.\text{new} = d_i.\text{dest} + 1$. As a result, the query result of $\text{COUNT}(\text{dest})$ equals $\text{SUM}(\text{new}) - \text{SUM}(\text{dest})$, effectively transforming the count query into two sum queries. Based on this, the average query is implemented using a sum query and a count query.

Except for aggregating queries, a query may require the original data result, for example, $\text{SELECT amt FROM TABLE T WHERE name} = n$, and the result $R = S_n$. For brevity, we refer to such a query as a select query. Taking an equivalent query as an example, the proof of a select query is generated by finding an item such that $\text{item.key} = n$, and the proof $\pi = (\text{item}, w_{\text{sub}})$. To verify R , check: 1) $\text{item.key} = n$; 2) $\text{isSub}(\text{item}, \delta_D)$ outputs 1; and 3) $\text{Acc}(S_n, G_1) = \text{item}_A.\text{val}$.

C. Index Combination

A query may consist of multiple conditions and multiple destinations. As long as there is a single-condition and single-destination index for each pair of destination and condition in the query, the query can be executed and verified by combining those indexes. We denote this method as index combination.

1) *Condition Combination*: Combining indexes with the same destination enables multi-condition queries. Assuming a query $\text{SELECT SUM}(\text{dest}) \text{ FROM TABLE T WHERE op}(\text{cond}_i) (i \in [1, k])$, where dest is a column, cond_i is an equivalent or range condition, and op indicates a logic operation (conjunction, disjunction, or hybrid) on those conditions.

- Query. For each condition cond_i , if a single-condition index exists, obtain a subset R_i and proof π_i through the “SELECT

name	bank	rate	amt
Alice	Bank1	4.3	85
Bob	Bank1	4.7	90
Alice	Bank2	5.1	70
Carol	Bank3	4.9	95
Bob	Bank2	4.8	60

tier index [(name, bank, rate)→amt]			
δ	Alice, δ_A	Bank1, δ_{A1}	4.3, Acc({85})
		Bank2, δ_{A2}	5.1, Acc({85,70})
	Bob, δ_B	Bank1, δ_{B1}	4.7, Acc({90})
		Bank2, δ_{B2}	4.8, Acc({90,60})
	Carol, δ_C	Bank3, δ_{B3}	4.9, Acc({95})

Fig. 4. A Tier Index on Table I.

dest where cond_i ” query, compute $f_{R_i} = \text{Acc}(R_i, G_1)$. Then, the result set $S_R = \text{op}(R_1, \dots, R_k)$, the result is $R = \text{sum}(S_R)$, and the query proof is $\pi = (\{\pi_i, f_{R_i}\}_{i \in [1,k]}, f_R, \pi_{\text{op}}, w_{\text{sum}})$, where $f_R = \text{Acc}(S_R, G_1)$, $\pi_{\text{op}} = \text{ProveOp}(R_1, \dots, R_k, \text{op}, S_R)$, $w_{\text{sum}} = \text{ProveSum}(S_R)$.

- Verify. 1) For each cond_i , run the first three steps of VerifyEq or VerifyRange to check the correctness of f_{R_i} ; 2) verify the correctness of set operation through $\text{IsOP}(f_{R_1}, \dots, f_{R_k}, \text{op}, f_R, \pi_{\text{op}})$; 3) verify the sum result through $\text{IsSum}(R, f_R, w_{\text{sum}})$. If all three checks pass, the result is correct.

Handle duplicated values: The presupposition for the condition combination is that there are no duplicated values in the destination column. For example, suppose there exists $d_6 = (\text{David}, \text{Bank1}, 70, 4.2)$ in Table I. When querying $\text{SELECT SUM(amt) FROM T WHERE name = Alice AND rate BETWEEN 4.0 and 4.5}$, the correct result should contain only $d_1.\text{amt}$. However, when querying on $[\text{name} \rightarrow \text{amt}]$ and $[\text{rate} \rightarrow \text{amt}]$, the results are $\{d_1.\text{amt}, d_3.\text{amt}\} = \{85, 70\}$ and $\{d_1.\text{amt}, d_6.\text{amt}\} = \{85, 70\}$, respectively, leading to an intersection result that is incorrect. We use two methods to handle this issue. On one hand, the index creation phase can split the data table into multiple tables to ensure that no duplicate values exist within the destination column of each table. On the other hand, if duplicates still exist after splitting, the corresponding contract records those data entries. Subsequently, during querying, the contract can check for the existence of duplicate values based on the query conditions and subtract them from the final results.

Here we describe how duplicates are subtracted from final results. Following the above example, $d_6 = (\text{David}, \text{Bank1}, 70, 4.2)$ introduces duplicate values in Table I. During the contract deploy phase, the deployer stores $\text{Dup}[70] = [(d_3, \text{Alice}, 70, 5.1), (d_6, \text{David}, 70, 4.2)]$ in the contract. During contract execution phase, if the database server returns a SUM result of 155, the contract detects that $d_3.\text{name} = \text{Alice}$ and $d_6.\text{rate}$ falls between 4.0 and 4.5 within $\text{Dup}[70]$. This identifies an intersection result that erroneously includes $\text{amt} = 70$. Then, the contract subtracts 70 from the aggregation, yielding the correct sum of 85.

Handling duplicate values introduces additional overhead. Let m be the number of split tables and $|\text{Dup}|$ be the number of duplicated entries. With table splitting, index construction and query cost scale by a factor of m , resulting in $O(m \cdot C_{\text{build}})$ for index construction and $O(m \cdot C_{\text{query}})$ for query, where C_{build} and C_{query} represent the initial cost without duplicated values. With on-chain duplicate record keeping, the contract incurs $O(|\text{Dup}|)$ extra storage and up to $O(m \cdot |\text{Dup}|)$ verification cost per query.

In Section V-D, we show an optimized tier-index-based design that does not incur such overhead.

2) *Destination Combination:* A destination combination can be straightforwardly realized by separately querying each destination. For instance, given a “SELECT SUM(amt), SUM(rate) FROM TABLE T WHERE name=n” query, we can acquire its result and proof by querying “SELECT SUM(amt)” and “SELECT SUM(rate)” respectively.

D. Multi-Cond Multi-Dest Index

Although the index combination works, it is not efficient because both condition combinations and destination combinations incur additional verification costs. Specifically, the condition combination requires set operation proofs, which brings additional pairing checks during verification (see Section III-B). For a destination combination, all destinations share the same condition but need to be verified separately, leading to wasted verification efforts. To tackle this problem, we propose the multi-cond multi-dest index to reduce the verification cost.

1) *Multi-Cond Index:* For multiple equivalent conditions, we can use a joint key or the tier index to construct multi-cond index. Suppose the value sets of k equivalent indexes are $V_1, \dots, V_k$, and $\prod_{i=1}^k |V_i|$ is less than the maximum value of the dictionary key, we can simply apply *join* on their values to generate dictionary items. For instance, for each $v_1, \dots, v_k \in V_1 \times \dots \times V_k$, generate key = $\text{Hash}(v_1 \parallel \dots \parallel v_k)$ and subset $S = \{d_i.\text{dest}\}_i$ for i s.t. $d_i.\text{col}_i = v_i$. Then, we have item = (key, $f_S, \text{next}(\text{key})$). Subsequently, to query $\text{SELECT dest FROM TABLE T WHERE col}_1 = v_1 \text{ AND } \dots \text{ AND col}_k = v_k$, the proof is similar to that of single-cond equivalent query.

When $\prod_{i=1}^k |V_i|$ is large, we use a tier index to enable efficient multi-condition equivalent queries. Algorithm 1 constructs the tier index by decomposing a k -condition space into nested authenticated dictionaries, where each tier corresponds to one condition attribute. The first tier indexes the values of the first condition; the second tier indexes the values of the second condition given a fixed first-condition value; and this pattern continues until the final tier indexes the values of the k th condition under a fixed prefix of the previous $k-1$ conditions. Consequently, each path in the tier index uniquely corresponds to one full condition combination $(v_1, v_2, \dots, v_k)$, allowing the verifier to authenticate a multi-condition result through a single path traversal rather than intersecting separate proofs.

Given a condition list $\text{AND}[\text{cond}_1, \dots, \text{cond}_k]$, the tier index creation process first obtains subsets $S_{v_1 \dots v_k} = \{d_j.\text{dest}\}_j$ for j s.t. $d_j.\text{cond}_i = v_i$. Intuitively, Algorithm 1 builds the index from the bottom tier upward. For every full condition combination $(v_1, v_2, \dots, v_k)$, it first computes the accumulator of the corresponding subset of results and creates a bottom-tier item using v_k as the key and the subset’s accumulator as the value (lines 7-11). These bottom-tier items together form a dictionary $D_{v_{k-1}}$ representing the last condition under the fixed prefix $(v_1, v_2, \dots, v_{k-1})$. Once this dictionary is completed, the algorithm computes its digest $\delta_{v_{k-1}}$ and inserted it into the upper-tier dictionary $D_{v_{k-2}}$ as the value associated with the key v_{k-1} (lines 12-14). The same process is repeated upward

Algorithm 1: Tier Index Construction Process.

input : All subsets $S_{v_1 \dots v_k}$
output: A tier index with digest δ_D

```

1 let  $D$  be an empty dictionary
2 for each  $v_1$  in  $V_1$  do
3   let  $D_{v_1}$  be an empty dictionary
4   ...
5   for each  $v_{k-1}$  in  $V_{k-1}$  do
6     let  $D_{v_{k-1}}$  be an empty dictionary
7     for each  $v_k$  in  $V_k$  do
8        $f_k \leftarrow \text{Acc}(S_{v_1 \dots v_k})$ 
9        $\text{item}_{v_k} \leftarrow (v_k, f_k, \text{next}(v_k))$ 
10      append  $\text{item}_{v_k}$  to  $D_{v_{k-1}}$ 
11    end
12     $\delta_{v_{k-1}} \leftarrow \text{Acc}(D_{v_{k-1}}, G_1)$ 
13     $\text{item}_{v_{k-1}} \leftarrow (v_{k-1}, \delta_{v_{k-1}}, \text{next}(v_{k-1}))$ 
14    append  $\text{item}_{v_{k-1}}$  to  $D_{v_{k-2}}$ 
15  end
16  ...
17   $\delta_{v_1} \leftarrow \text{Acc}(D_{v_1}, G_1)$ 
18   $\text{item}_{v_1} \leftarrow (v_1, \delta_{v_1}, \text{next}(v_1))$ 
19  append  $\text{item}_{v_1}$  to  $D$ 
20 end
21  $\delta_D \leftarrow \text{Acc}(D, G_1)$ 

```

(lines 4, 16). Each upper-tier item stores the digest of its child dictionary, until the top dictionary D is constructed (lines 17-19). The digest δ_D of the top dictionary is the digest of the entire multi-condition index (line 21). For example, Fig. 4 illustrates a 3-tier index on Table I that supports conjunction queries on $[[\text{name}, \text{bank}, \text{rate}] \rightarrow \text{amt}]$. $\text{item}_{\text{Alice}}$ in the first tier does not directly store a subset of results. Instead, its value is the digest of a child dictionary that indexes the next condition, including Bank1 and Bank2. Similarly, each item at the second tier stores its child dictionary for the third condition. Only the bottom tier stores accumulators of actual result subsets. In this way, the tier index organizes the multi-condition space as a nested dictionary structure.

The tier index supports a conjunction query on the conditions, i.e., $\text{SELECT SUM}(\text{dest}) \text{ FROM TABLE T WHERE } \text{col}_1 = v_i \text{ AND } \dots \text{ AND } \text{col}_k = v_k$. The proof consists of k dictionary items ($\text{item}_{v_1}, \dots, \text{item}_{v_k}$) and their subset witnesses. To verify the result, the verification process checks: 1) the sum witness, 2) whether each item key satisfies the condition value, and 3) whether each item belongs to the dictionary. For item_{v_1} , the dictionary digest δ_D is stored by the verifier, and for item_{v_i} ($i > 1$), the dictionary digest is the value of $\text{item}_{v_{i-1}}$.

In addition, the bottom tier (cond_k) can be a range condition. To achieve this, when computing f_k in Algorithm 1 (line 8), the input set of Acc should be a union of multiple subsets whose v_k is not greater than the current v_k . It should be noted that the tier index does not support inserting a range query in a non-bottom tier, nor does it support multiple range queries.

This structure enables efficient multi-condition verification because each query determines a unique path. The verifier only needs to check membership along this path and then verify the final proof on the bottom-tier accumulator, which avoids set intersections across multiple single-condition results. Consequently, the tier index remains immune to the challenges posed by duplicate values and avoids the associated

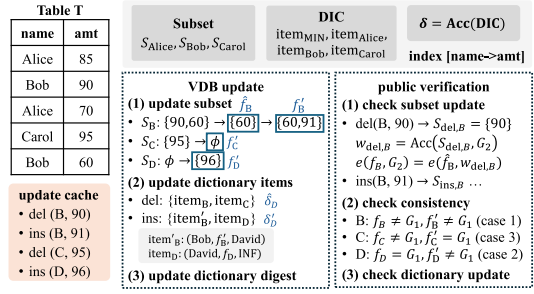


Fig. 5. VDB Update and Public Verification.

processing overhead, providing a more efficient solution for complex multi-condition queries.

2) **Multi-Dest Index:** The motivation to create a multi-dest index is to reduce the dictionary item verification cost when multiple destinations are requested. The method is to record multiple subset accumulators in each dictionary items that related to multiple destinations, allowing them to be verified in one item verification. For instance, given two indexes $[\text{name} \rightarrow \text{amt}]$ and $[\text{name} \rightarrow \text{rate}]$, for $\text{name} = n$, they separately have $\text{item}_n = (n, f_{S_{n1}}, \text{next}(n))$ and $\text{item}_n = (n, f_{S_{n2}}, \text{next}(n))$, where $S_{n1} = \{d_i.\text{amt}\}$, $S_{n2} = \{d_i.\text{rate}\}$ ($d_i.\text{name} = n$). To create a multi-dest index $[\text{name} \rightarrow [\text{amt}, \text{rate}]]$, for each $n \in V_{\text{name}}$, let $\text{item}_n.\text{val} = (f_{S_{n1}}, f_{S_{n2}})$. As a result, the correctness of $f_{S_{n1}}$ and $f_{S_{n2}}$ can be determined through one dictionary item check.

VI. DISPUTABLE UPDATE

The disputable update mechanism reduces the contract cost to verify the updated digest with the help of public verification and contract dispute. The update process consists of four phases: update cache, VDB update, public verification, and contract dispute. Fig. 5 illustrates a simple example of the VDB update and public verification processes on Table I.

Step 1. Update cache: When the contract generates an update demand, it caches the update in the contract storage, while the VDB remains unchanged. For convenience, update demands are divided into delete and insert types, and a modification of an existing data item is split into one delete and one insert process. When the contract verifies query results, it dynamically updates the result according to the cache. For instance, given the table and update cache in Fig. 5 and a query instruction $\text{SELECT SUM}(\text{amt}) \text{ FROM T WHERE name} = \text{'Bob'}$, the query result obtained from the VDB should be 150. According to the update cache $\text{del}(\text{B}, 90)$ and $\text{ins}(\text{B}, 91)$, the contract can update the result to $150 - 90 + 91 = 151$.

Step 2. VDB update: When the cache reaches its upper limit, the VDB handles the update to generate a new digest along with the update proof. As shown in Fig. 5, the update process consists of two steps: updating the subset and updating the dictionary. Here, we take the equivalent index as an example to detail the update process, and it is straightforward to detail other indexes, including range or multi-cond indexes, using the same principle.

1) **Update subset.** This step updates the subset of each condition value $n \in V_{\text{cond}}$. Specifically, let $S_{\text{del}, n} = \{d_i.\text{dest}\}$ and $S_{\text{ins}, n} = \{d_i.\text{dest}\}$ ($d_i.\text{cond} = n$) separately denote the

data objects that should be deleted from and inserted into S_n . Then, for each n such that one of $S_{\text{del},n}$ or $S_{\text{ins},n}$ is non-empty, computing $\hat{S}_n = S_n \setminus S_{\text{del},n}$, $S'_n = \hat{S} \cup S_{\text{ins},n}$, $\hat{f}_n = \text{Acc}(\hat{S}_n, G_1)$, and $f'_n = \text{Acc}(S'_n, G_1)$.

- 2) *Update dictionary items*: This step updates dictionary items that are affected by previous steps and updates the digest. Specifically, for each n that exists in the deletion or insertion set, update the related dictionary item. There are three cases for item updates: updating the item value, inserting the item, and deleting the item, depending on whether n exists before and after the update. In addition, deleting or inserting an item leads to the change of item keys of its adjacent items. Subsequently, the VDB generates the item deletion set D_{del} and insertion set D_{ins} containing items to be deleted and inserted respectively.
- 3) *Update dictionary digest*: Similar to element delete and insert, the VDB first handles the items D_{del} to delete, getting $\hat{\delta}_D$, and then inserts items D_{ins} , generating δ'_D . Then, the VDB publishes the update proof which consists of three parts: 1) the subset digests $\pi_{\text{subset}} = \{(f_n, \hat{f}_n, f'_n)\}$ for each n that is affected by the update; 2) the item deletion and insertion sets $\pi_{\text{item}} = (D_{\text{del}}, D_{\text{ins}})$ and their accumulators $w_{\text{itemDel}} = \text{Acc}(D_{\text{del}}, G_2)$ and $w_{\text{itemIns}} = \text{Acc}(D_{\text{ins}}, G_2)$; 3) the dictionary digests $(\delta_D, \hat{\delta}_D, \delta'_D)$.

Finally, the VDB uploads $(w_{\text{itemDel}}, w_{\text{itemIns}}, \hat{\delta}_D, \delta'_D)$ to the contract. The contract checks whether $e(\delta_D, G_2) = e(\hat{\delta}_D, w_{\text{itemDel}})$ and $e(\hat{\delta}_D, w_{\text{itemIns}}) = e(\delta'_D, G_2)$. If the check fails, the contract directly reject the updated digest; otherwise, it waits for the public verification and dispute.

Step 3. Public verification: Each entity can act as a verifier ($\mathcal{V}$) to verify the correctness of δ'_D based on the update proof.

- 1) *Check subset update*: $\mathcal{V}$ computes the element deletion and insertion sets $(S_{\text{del},n}, S_{\text{ins},n})$ based on the update cache. If either of $(S_{\text{del},n}, S_{\text{ins},n})$ is nonempty, $\mathcal{V}$ computes $w_{\text{del},n} = \text{Acc}(S_{\text{del},n}, G_2)$ and $w_{\text{ins},n} = \text{Acc}(S_{\text{ins},n}, G_2)$, checks whether $e(f_n, G_2) = e(\hat{f}_n, w_{\text{del},n})$ and $e(\hat{f}_n, w_{\text{ins},n}) = e(f'_n, G_2)$.
- 2) *Check consistency between π_{subset} and π_{item}* : The purpose of this check is to ensure that the update of subsets has been correctly reflected in the dictionary items. For each $(f_n, \hat{f}_n, f'_n)$ tuple in π_{subset} , there are three cases with different check demands.
 - Case 1: $f_n \neq G_1, f'_n \neq G_1$. In this case, S_n is non-empty both before and after the update. Thus, there should be an item in D_{del} with $\text{key}=n$ and $\text{val}=f_n$. Meanwhile, there should be an item in D_{ins} with $\text{key}=n, \text{val}=f'_n$.
 - Case 2: $f_n = G_1, f'_n \neq G_1$. In this case, S_n is empty before the update but non-empty after. This implies that a new item with $\text{key}=n$ and $\text{val}=f'_n$ is inserted into the dictionary. Suppose $\text{next}(n_1)=n$ and $\text{next}(n)=n_2$. In this case, the item with $\text{key}=n_1$ should also be updated by modifying its next field from n_2 to n . Thus, an item (n_1, f_{n_1}, n_2) should be contained in D_{del} , two items $(n, f_n, n_2), (n_1, f_{n_1}, n)$ should be contained in D_{ins} .

Notably, (n_1, f_{n_1}, n_2) also denotes the non-existence of item _{n} in the previous VDB version, preventing from inserting items with repeated keys.

- Case 3: $f_n \neq G_1, f'_n = G_1$. In this case, S_n is non-empty before the update but empty after. This implies that item _{n} is deleted from the dictionary. Suppose $\text{next}(n_1)=n$ and $\text{next}(n)=n_2$, D_{del} should contain (n, f_n, n_2) and (n_1, f_{n_1}, n) , while D_{ins} should contain (n_1, f_{n_1}, n_2) . The two items contained in D_{del} also proves the existence of item _{n} in the previous VDB version, preventing from deleting a non-existing item. During the check, if a required item does not exist in D_{del} or D_{ins} , the check fails since there are items that should be modified but are not. After handling all elements in π_{subset} , if there are items in π_{item} that are not used, the check is also failed, due to the fact that there are items that are modified but should not.
- 3) *Check dictionary update*: This step checks that the items in D_{del} are contained in the previous version of VDB and are correctly deleted, while D_{ins} are correctly inserted into the new version. Since the pairing checks are done by the contract, the verifiers only check whether $w_{\text{del},n} = \text{Acc}(S_{\text{del},n}, G_2)$ and $w_{\text{ins},n} = \text{Acc}(S_{\text{ins},n}, G_2)$.

Step 4. Contract dispute: If all checks pass, the verifier commits a correct flag to the contract; otherwise, commits an incorrect flag to the contract. Afterwards, the contract will accept the update when it collects sufficient correct flags or reject the update when it collects sufficient incorrect flags. The threshold for determining a ‘sufficient’ number of flags is a configurable parameter determined by the contract deployer. For instance, a deployer may require a simple majority or a Byzantine-fault-tolerant supermajority of verifier votes to optimistically accept an update. This choice represents a trade-off regarding economic cost (verification rewards), processing latency (asking for more votes takes a longer time) and belief certainty (where a larger number of voters yields higher confidence).

Crucially, we emphasize that the threshold does not impact the underlying security (correctness) of the system. The voting mechanism is strictly for optimistic operation. Even if a malicious majority provides ‘correct’ flags for an invalid update, the update will not be finalized if at least one honest verifier submits a cryptographic proof of invalidity during the dispute window. Thus, the system’s security is guaranteed by the ‘1-of- N honest’ assumption for the dispute phase, while the voting threshold merely governs the efficiency of the optimistic path. The dispute process works as follows.

The verifier can generate a dispute that enables the contract to identify an incorrect update immediately at a low cost, similar to verifying a query result, thus accelerating the verification process. Specifically, if the first (subset update) or second (item update) check fails while the third check (digest update) succeeds, when querying the related $\text{cond} = n$, where n is the condition value of the failed f_n , the result related to δ'_D must be incorrect. In such cases, $\text{dispute} = (R, \pi_R, R', \pi'_R)$, where (R, π_R) and (R', π'_R) are query results and proofs generated based on δ_D and δ'_D , respectively. Upon receiving the dispute, the contract verifies R and R' . Then, it updates R according to

the update cache and checks whether the result equals R' . If not, the contract can determine the incorrectness of δ'_D and reject the update.

However, if the third step fails, it may be challenging to locate a query condition that can generate a dispute. In this case, the dispute should consist of the entire $\pi_{\text{item}} = (D_{\text{del}}, D_{\text{ins}})$, and the contract should compute the two accumulators. When π_{item} is large, such a dispute becomes impractical. Therefore, when the third step fails, it is generally sufficient to submit an incorrect flag without submitting a dispute; the only exception is when a verifier discovers that a wrong updated digest receives sufficient correct flags. In this case, a dispute must be submitted to help the contract identify the error.

Incentives for Verifiers: ContractDB utilizes a verifier committee to assist data update verification. This committee consists of registered participants who submit flags or disputes during the designated dispute window. In practice, this set can be instantiated from staked participants or derived directly from the blockchain's validator set. We sketch an incentive mechanism to ensure active participation. At initialization, data owners or contract deployers deposit a maintenance fee to ensure data availability, while database servers post a bond as a guarantee against malicious behavior.

Interested parties can register as verifiers by staking assets, similar to Proof-of-Stake (PoS) models. For each update, these fees are logically allocated to three goals: (1) if an update is verified without a successful dispute, a portion of the maintenance fee is distributed to honest verifiers; (2) if a dispute occurs, the database server's bond is redistributed to the verifier who submitted the valid cryptographic proof to dispute; and (3) verifiers providing flags inconsistent with the final outcome are penalized through staking slashing. Furthermore, these procedures are recorded on-chain, serving as a transparent service history. This reputation-based metric allows users to estimate a server's reliability, which in turn incentivizes servers to remain honest to protect their business interests and attract future customers.

VII. SECURITY ANALYSIS

As defined in Section IV-B, the security of ContractDB consists of correctness and consistency. The security insights provided in Section IV-C show that the correctness of ContractDB relies on the correctness of the VDB, while consistency is guaranteed by the update cache mechanism. Thus, in this section, we focus on the security of the proposed VDB scheme and update mechanism.

A. Security of VDB Query

As defined in Section IV-B, query security requires that only correct query results can pass verification. Our proposed VDB scheme consists of single-cond single-dest equivalent and range indexes, index combination, and multi-cond multi-dest indexes. Assuming there exists a prover and a verifier, we prove the correctness of each kind of index as follows.

Theorem 1. (Correctness of the result accumulator) Given a query that consists of one or multiple conditions, assume a result set $S_R = \{d_i.\text{dest}\}$ ($d_i.\text{cond}$ satisfying query conditions), let

f_R denote the accumulator of S_R , a verifier can determine the correctness of f_R .

Proof. For a query with condition $\text{cond}=n$, the verifier only accepts f_R satisfying $\text{item.val}=f_R$, $\text{item.key}=n$, and item exists in the dictionary with digest δ_D . Thus, to make an incorrect f'_R pass the verification, the prover must construct a subset witness proving that an item $(n, f'_R, \text{next}(n))$ exists in the dictionary with digest δ_D . According to the subset security of accumulator [34], it is impossible to forge a subset witness for a non-included element.

For a condition $\text{cond} \in [v_l, v_r]$, the verifier only accepts f_R satisfying that $e(f_R, \text{item}_1.\text{value}) = e(G_1, \text{item}_2.\text{value})$, $\text{item}_1.\text{key} < v_l \leq \text{item}_1.\text{next}$, $\text{item}_2.\text{key} \leq v_r < \text{item}_2.\text{next}$, and both item_1 and item_2 are contained in the dictionary with digest δ_D . According to the security of set difference proved in [21], if $f'_R \neq f_R$, it is impossible to make $e(f'_R, \text{item}_1.\text{value})$ equals $e(G_1, \text{item}_2.\text{value})$. Thus, to make the verifier accept f'_R , the prover should succeed in one of the following cases: 1) construct a subset witness proving that an item item'_1 satisfying $e(f'_R, \text{item}'_1.\text{value}) = e(G_1, \text{item}_2.\text{value})$ exists in the dictionary with digest δ_D ; 2) construct a subset witness proving that an item item'_2 satisfying $e(f'_R, \text{item}'_2.\text{value}) = e(G_1, \text{item}_2.\text{value})$; or 3) constructing item'_1 and item'_2 and their subset witnesses simultaneously such that $e(f'_R, \text{item}'_1.\text{value}) = e(G_1, \text{item}'_2.\text{value})$. According to the subset security [34], the subset witness in any of these three cases is unable to be constructed.

For a multi-condition query based on multi-condition indexes, similar to that of single-cond queries, the correctness is guaranteed by subset security (which prevents the forging of items) and difference security (when the bottom tier is a range query).

For a multi-cond query based on index combination, suppose the result set of each condition is $(S_{R_1}, \dots, S_{R_k})$ whose accumulator is $(f_1, \dots, f_k)$. Then, the result set of the full query should be $S_R = \text{op}(S_{R_1}, \dots, S_{R_k})$ with an accumulator f_R . Since the verifier can determine the correctness of f_1 to f_k based on single-cond verification and determine the correctness of op according to ASO [34], the verifier can determine the correctness of f_R . $\square$

With f_R being verifiable, it is natural to prove the correctness of the query results. For a sum query, to make the verifier accept an incorrect sum result $\text{sum}(S'_R)$, the prover should construct a sum witness w_{sum} making $\text{IsSum}(\text{sum}(S'_R), f_R, w_{\text{sum}})$ outputs 1, where $f_R = \text{Acc}(S_R)$ and $S_R \neq S'_R$. This is proved to be impossible in [22]. For a select query that requesting the origin data, the verifier will accept an incorrect result R' only when $\text{Acc}(R') = f_R$ happens. Since $f_R = \text{Acc}(R)$, it is required that $\text{Acc}(R') = \text{Acc}(R)$, $R' \neq R$. This is proved to be impossible in [34].

The above proofs depend on the correctness of δ_D . ContractDB assumes that when a DApp is initially deployed, it contains a correct δ_D that is constructed based on the legacy data. This is a necessary prerequisite, as the trust boundary of a DApp should be defined at deployment and not extended backward without limit. When the DApp generates update demands on VDB, the correctness of δ_D is guaranteed by the security of VDB update, as follows.

B. Security of VDB Update

Theorem 2. (VDB Update Correctness) Given two index digests (δ_D, δ'_D) and the update demands including a deletion set $S_{\text{del}} = \{d_i\}$ and an insertion set $S_{\text{ins}} = \{d_i\}$, the public verification process can determine the correctness of δ'_D .

Proof. Intuitively, update correctness is decomposed into three checks that match the three steps of the public verification process. First, subset correctness ensures that each affected subset is updated exactly according to the declared deletion and insertion sets $(S_{\text{del}}, S_{\text{ins}})$. Second, dictionary item correctness ensures that the modified dictionary items are consistent with the subset updates verified in step 1. Third, dictionary digest correctness ensures the new digest δ'_D accurately reflects the application of the update to the previous dictionary.

The three-step verification process is interlinked; any incorrect update must violate at least one of these relations. Passing all three checks would require forging the underlying accumulator relations, which is computationally infeasible. We provide formal proofs for each of these three requirements.

Suppose a subset S_n should have been updated to S'_n but is incorrectly updated to S''_n with a resulting accumulator f''_n . To convince the verifier to accept f''_n , the prover would need to forge an intermediate value $\hat{f}'_n$ such that $\hat{f}'_n \neq \hat{f}_n$, $e(f_n, G_2) = e(\hat{f}'_n, w_{\text{del},n})$ and $e(\hat{f}'_n, w_{\text{ins},n}) = e(f''_n, G_2)$. This is computationally infeasible under the security assumptions of the underlying accumulator schemes. Therefore, given a tuple $(f_n, \hat{f}_n, f'_n)$, it is ensured that f'_n represents a subset $S'_n = S_n - S_{\text{del},n} + S_{\text{ins},n}$.

For the item update, the consistency check ensures that the deleted and inserted items in $(D_{\text{del}}, D_{\text{ins}})$ are consistent with the previously verified subset updates. Let f_p and f'_p denote the old and new subset accumulator values and let $\hat{f}_p$ denote the intermediate value after deletion. The consistency check guarantees that the subset's accumulator value becomes $\hat{f}_p$ after deleting the dictionary item in D_{del} , and reaches f'_p after insertion. In other words, in the tuple $(f_p, \hat{f}_p, f'_p)$, f_p and f'_p are included in the previous and last versions of the dictionary respectively, implying that D_{del} and D_{ins} are correctly related to the subset update.

Finally, since the digest update is similar to the subset update, it ensures that δ'_D represents a dictionary $D' = D - D_{\text{del}} + D_{\text{ins}}$. In conclusion, only a correct δ'_D can be accepted by a verifier. $\square$

With update correctness, as long as the number of dishonest verifiers is less than the threshold of correct flags required for the contract to accept the update, the update process remains secure. In addition, even with an excessive number of dishonest verifiers, the contract can identify the incorrectness with the help of the dispute mechanism. Although the dispute may incur significant costs (e.g., taking π_{item} as the dispute), our proposed ContractDB guarantees that in the worst-case scenario, i.e., when there is only one honest verifier in the system, security will persist.

VIII. PERFORMANCE ANALYSIS

In this section, we evaluate the performance of our proposed ContractDB protocol. We first evaluate the index creation, query,

and verification costs of single-cond single-dest indexes for both equivalent and range indexes. Next, we test the query and verification costs of multi-cond queries based on index combinations. Finally, we evaluate the costs of multi-cond indexes and compare them with index combinations.

A. Implementation

We implement the VDB construction using Golang and the contract verification using Solidity. We utilize the bn254 curve provided by gnark-crypto for bilinear pairing, as it is currently the only curve supported by Solidity. The contract verification relies on the precompiled contracts with addresses 0x6 to 0x8, which separately implement curve addition, curve multiplication, and curve pairing on bn254. Both creation and query costs are represented by the CPU runtime of the VDB server, which is a cloud server with 52 processes and 64 GB of RAM. The verification cost is evaluated based on the gas consumption of the Ethereum contract to complete the verification. The smart contract runs on a local Ethereum testnet generated by ganache-cli.

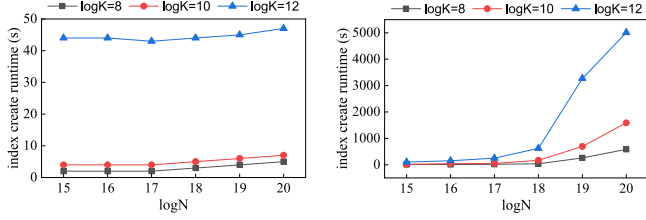
Data generator: Our experiments are conducted on a SQL table consisting of 7 columns, each occupying 8 bytes. Among the columns, three serve as equivalent conditions, three act as range conditions, and the last one serves as the query destination. The number of possible values for each condition column ($k = |V|$) is set to $k \in \{2^8, 2^{10}, 2^{12}\}$ for both equivalent and range conditions, while the destination column contains no duplicate values. The total number of lines in the table (N) varies from 2^{15} to 2^{20} .

B. Theoretical Analysis

Let $n = N/k$ denote the average number of data objects per condition value, and $|R|$ denote the size of the result set. $|R|$ equals n for an equivalent query and is a multiple of n for a range query. Let $C_{A(n)}$ denote the CPU cost to compute the accumulator of an n -element set, (G_e, G_m) denote the gas cost of pairing checks and curve multiplications.

1) Equivalent Index: The index creation process of an equivalent index consists of three parts: computing the subset accumulator, computing the item subset witness, and computing the dictionary digest. The first part incurs a computational cost of $k \cdot C_{A(n)}$, as there are k subsets with an average size of n . The second part requires $k \cdot C_{A(k-1)}$ operations based on the ProveSub procedure, and the third part involves $C_{A(k)}$ operations for processing k items in total. We approximate $k \cdot C_{A(k-1)} + C_{A(k)}$ as $k \cdot C_{A(k)}$. Therefore, the total cost for proof creation is $k \cdot C_{A(n)} + k \cdot C_{A(k)}$. The query process requires $C_{A(|R|-1)} + C_{A(|R|-2)}$ operations to generate the sum witness, which we simplify to $2C_{A(|R|)}$. The verification process consists of a subset check and a sum check. The former requires one curve multiplication and one pairing, while the latter requires two curve multiplication and two pairings. Thus, the total cost is $2G_m + 3G_e$.

2) Range Index: The range index subset contains data objects satisfying $d_i.\text{cond} \leq v$, whereas the equivalent index contains objects where $d_i.\text{cond} = v$. Consequently, the computational



(a) Runtime to Create Eq Index (b) Runtime to Create Range Index

Fig. 6. Setup Cost of Single-cond single-dest Indexes.

cost for generating the subset accumulators of the range index is $\sum_{i=1}^k C_{A(i,n)}$, rather than $k \cdot C_{A(n)}$. The query cost remains $2C_{A(|R|)}$ for generating the sum witness. The verification process involves checking two subset witnesses (each requiring one scalar multiplication in $\mathbb{G}$ and one pairing), one sum witness (requiring two scalar multiplications and two pairings), and one set difference (requiring one pairing). The total verification cost is therefore $4G_m + 5G_e$.

3) *Index Update*: Suppose there are t data entries with difference condition values to be updated. The update cost of an equivalent index consists of updating t subsets, t subset witnesses, and the dictionary digest, leading to a cost of $t \cdot C_{A(n)} + t \cdot C_{A(k-1)} + C_{A(k)}$. For a range index, if a subset S_{v_0} need to be updated, all subsets S_{v_i} where $v_i > v_0$ must also be updated. Thus, in the worst case, the update cost is similar to creating a new one.

4) *Other Queries*: As explained in Section V-B3, except for sum queries, the proposed VDB also supports other queries including max, min, count, average, and the select query requesting original data. The cost of the max-min query is similar to adding an additional range condition to the original conditions. The cost of the count query is twice that of the sum query, since it is executed as two separate sum queries. The average query includes one sum query and one count query. Unlike the sum query, the verification process of a select query does not check sum witnesses. Instead, it computes the accumulator of the result set. This requires the contract to compute the characteristic polynomial of the result set and then compute the accumulator. Such a procedure requires $O(|R|^2)$ integer multiplications and $O(|R|)$ scalar multiplications and additions. Considering the above results, it is not recommended to query the original data objects in ContractDB.

C. Single-Cond Single-Dest Indexes

1) *Index Creation*: The index creation cost is affected by the index type (equivalent or range), the dataset size (N), and the number of possible values of the condition column (k). We evaluate the index creation cost of equivalent and range indexes for N varying from 2^{15} to 2^{20} and $k \in \{2^8, 2^{10}, 2^{12}\}$. Results are shown in Fig. 6. As Fig. 6(a) indicates, creating an equivalent index takes less than a minute, while Fig. 6(b) shows that creating a range index may take more than one hour when $N=2^{20}$ and $k=2^{12}$.

2) *Query and Verification*: Based on the indexes, we evaluate the query and verification costs for different conjunction queries

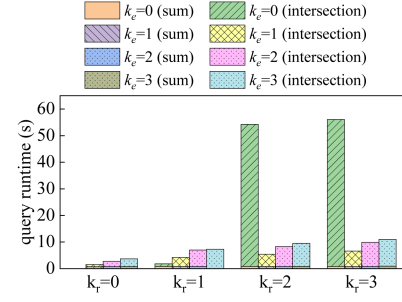


Fig. 7. Query Cost of Index Combination.

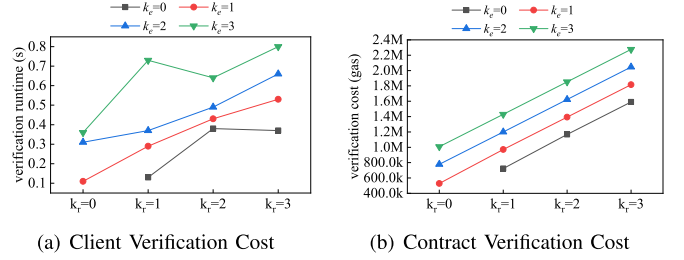


Fig. 8. Verification Cost of Index Combination.

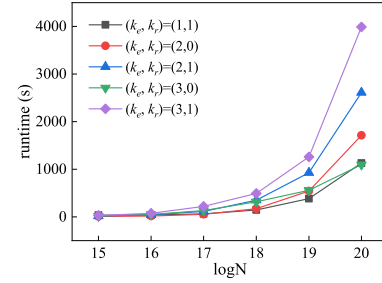


Fig. 9. Runtime of Multi-cond Indexes Creation.

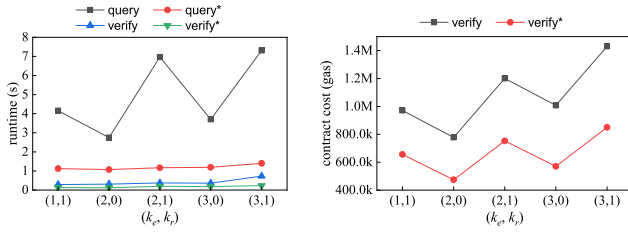
with $N = 2^{20}$. We write k_e and k_r to denote the number of equivalent and range conditions in a query. Then, we test the query and verification cost of conditions consists $k_e \in \{0, 1, 2, 3\}$ equivalent queries and $k_r \in \{0, 1, 2, 3\}$ range queries, and results are shown in Fig. 7 and Fig. 8.

For query cost, we separately record the runtime for generating the sum proof and the intersection proof, with the total runtime being the sum of both. Fig. 7 shows that in some cases, generating an intersection proof may take around 50 seconds, while in most cases, several seconds are sufficient.

For verification, Fig. 8(a) shows that verification time for a client to verify the results is less than one second, while Fig. 8(b) displays the gas cost for a contract to verify query results. For a 6-condition intersection query with $(k_e, k_r)=(3, 3)$, the verification cost is around 2.4 million gas and is independent of N .

D. Multi-Cond Index

The index creation cost is dependent on the condition numbers and table size. Thus, we evaluate the CPU runtime of creating multi-cond index for N varying from 2^{15} to 2^{20} , and $(k_e, k_r) \in \{(1, 1), (2, 0), (2, 1), (3, 0), (3, 1)\}$. The results in Fig. 9 show



(a) Query and Verification Runtime (b) Contract Verification Cost (gas)

Fig. 10. Query and Verification Costs of Multi-Cond Indexes.

that the creation cost of multi-cond indexes is around 1000 seconds less than creating separate single-cond indexes.

Based on the multi-condition indexes, we evaluate the query and verification costs of multi-condition queries, with the results shown in Fig. 10. The superscript “*” denotes results on multi-condition indexes. Results in Fig. 10(a) show that the time for both query and verification of multi-condition indexes are lower than those of index combinations, while results in Fig. 10(b) demonstrate that multi-condition indexes reduce gas costs by half compared to index combinations.

E. Interpretation of Performance Results

To bridge the gap between raw experimental data and practical system deployment, we synthesize our performance findings into actionable insights regarding computational overhead and architectural trade-offs.

In ContractDB, the client-side verification overhead is acceptable for mobile devices. First, the storage requirement is minimal. The required storage is small because the client only needs the first few public parameters used by the verification functions, rather than the entire public parameter set. Second, the computational workload consists of a limited number of bilinear pairing checks, group operations, and hash functions. They are standard cryptographic operations and can be executed on mobile devices with appropriate elliptic-curve cryptographic libraries. On modern smartphones, the elliptic-curve-based operations range from sub-millisecond point additions to millisecond-scale bilinear pairings [36], performance levels comparable to the CPUs used in our experimental benchmarks, ensuring that the total verification time remains well within the typical energy and latency budget of mobile users.

Using single index combination or multi-condition index is a trade-off between flexibility and cost. Index combination is more flexible for diverse query patterns because it reuses independently constructed single-condition indexes and does not require pre-building a dedicated index for every possible condition combination. However, it incurs extra proof-generation and verification cost when combining multiple indexes. In contrast, the optimized multi-condition index is more efficient for fixed multi-condition workloads and achieves lower query time and gas cost. The drawback is that such indexes must be built and maintained for predefined condition combinations, making them less flexible.

Range indexes are significantly more computationally intensive to build than equality indexes because they are constructed

over cumulative subsets of ordered keys (which is the inherent computational bottleneck), as detailed in Section VIII-B. While this is primarily a one-time setup cost, it is important to note that any subsequent updates altering the underlying data distribution may necessitate a partial or full rebuild of the proof structure.

The experimental results demonstrate that range index construction is efficient for datasets whose size is up to 2^{18} , with creation times consistently within a few minutes (Fig. 6(b)). However, we observe that as both the data size (N) and the number of possible values (k) increase—specifically beyond $N = 2^{18}$, $k = 2^{12}$ —the construction time increases substantially. This is attributed to the cumulative nature of the proof structure. Consequently, ContractDB is optimally suited for large-scale datasets bounded within this practical threshold. Beyond this range, the performance overhead suggests that future research should focus on alternative indexing architectures or hierarchical proof-decomposition mechanisms to further enhance scalability for massive datasets.

F. Discussion on the Accumulator

In our proposed construction, the bilinear accumulator is utilized to facilitate both authenticated set operations and authenticated dictionaries. While the accumulator remains fundamental to the former, the authenticated dictionary component may be substituted with more computationally efficient alternatives, such as a Merkle Hash Tree [2] or an Aggregatable Vector Commitment [37]. Table III provides a comparison of alternative data structures for authenticated dictionaries.

Merkle Hash Tree (MHT)-based authenticated dictionaries require no trusted setup and rely primarily on hash computations, resulting in low construction overhead. However, since membership proofs consist of root-to-leaf authentication paths, the proof size is $O(\log N)$. Furthermore, MHTs also present challenges for data updates, as nodes update needs non-trivial adjustments of the tree structure.

Vector commitments (VCs) rely on group operations, leading to higher construction and verification costs compared to hash-based trees. VCs provide constant-size openings, ensuring a proof size of $O(1)$. Their algebraic structure is well-suited for both updates and proof combination, offering robust dynamic and aggregation support.

Bilinear accumulators are pairing-based and incur higher construction cost compared to the other two. They provide constant-size membership witnesses, maintaining a proof size of $O(1)$. They inherently support insertions and deletions, providing strong dynamic support. Their key strength is support for authenticated set operations (ASO), which naturally fits subset authentication and other set-oriented verification.

In summary, an MHT-based approach is most suitable for relatively static environments where simplicity is the priority. In contrast, vector commitments and bilinear accumulators are preferable for dynamic settings requiring succinct proofs. Bilinear accumulators excel in handling complex set operations. Since ContractDB relies more heavily on authenticated set operations, we choose bilinear accumulators as the basis in the authenticated dictionary design for the ease of integration.

TABLE III
COMPARISON OF CANDIDATE ADS FOR DESIGNING AUTHENTICATED DICTIONARIES

ADS	Setup/Assumptions	Construction Cost	Proof Size	Verification Operations	Dynamic	Key Strength
Merkle Hash Tree	No Trusted Setup	Low	$O(\log N)$	Hash Checks	Limited	Simplicity
Vector Commitment	Public Parameters	High	$O(1)$	Group Operations	Strong	Proof Aggregation
Bilinear Accumulator	Pairing-based	High	$O(1)$	Pairing Checks	Strong	ASO Support

IX. CONCLUSION

In this paper, we proposed a new architecture that leverages external verifiable databases (VDB) to store and query data, to facilitate the large legacy data storage and processing with blockchain decentralized applications (DApps). We proposed ContractDB to securely instantiate such an architecture where we design authenticated indexes to ensure that the query results can be verified by smart contracts. Specifically, we designed a new VDB construction that utilizes authenticated dictionary and authenticated set operation technologies to enable public update verification and efficient query verification. Based on this, we designed a disputable update mechanism that supports efficient update verification. We provided a comprehensive security proof to demonstrate the security of ContractDB and conducted thorough experiments to evaluate its performance. The results show that ContractDB can integrate large datasets with complex conditional queries at an acceptable gas cost.

ContractDB complements existing blockchain scaling paradigms, including rollups, Data Availability (DA) layers, and database-assisted blockchains. While rollups and DA layers successfully prioritize transaction throughput and data accessibility, they remain fundamentally constrained in their ability to process and store large-scale, complex contract data. Conversely, database-assisted blockchains—such as Hyperledger Fabric utilizing CouchDB—leverage external databases to improve storage and query efficiency; however, these typically operate under permissioned models where the correctness of queries and updates lacks public verifiability. ContractDB bridges these gaps through a hybrid storage model that offloads data-intensive storage and complex query processing to external Verifiable Databases (VDBs). This architecture allows ContractDB to function as a vital extension of the blockchain scaling stack, seamlessly integrating the high-performance capabilities of off-chain database management with public, on-chain verification. In the future, we aim to integrate privacy preservation methods to protect the privacy of database updates.

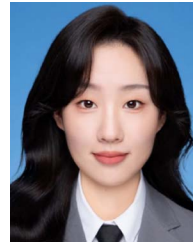
REFERENCES

- [1] V. Buterin, "Ethereum: A next-generation smart contract and decentralized application platform," 2014, Ethereum yellow paper. Accessed: Dec. 2025. [Online]. Available: https://ethereum.org/content/whitepaper/whitepaper-pdf/Ethereum_Whitepaper_-_Buterin_2014.pdf
- [2] S. Nakamoto, "Bitcoin: A peer-to-peer electronic cash system," 2008, Bitcoin white paper. Accessed: Dec. 2025. [Online]. Available: <https://bitcoin.org/bitcoin.pdf>
- [3] W. Gavin, "Ethereum: A secure decentralised generalised transaction ledger (Berlin version)," 2022, Ethereum yellow paper. Accessed: Dec. 2025. [Online]. Available: <https://ethereum.github.io/yellowpaper/paper.pdf>
- [4] N. Shrestha, R. Shrothrium, A. Kate, and K. Nayak, "Sailfish: Towards improving the latency of DAG-based BFT," in *Proc. IEEE Symp. Secur. Privacy*, 2025, pp. 1928–1946.
- [5] B. Arun, Z. Li, F. Suri-Payer, S. Das, and A. Spiegelman, "Shoal: High throughput DAG BFT can be fast and robust," in *Proc. USENIX Symp. Networked Syst. Des. Implementation*, 2025, pp. 813–826.
- [6] L. Jia, Y. Liu, K. Wang, and Y. Sun, "Estuary: A low cross-shard blockchain sharding protocol based on state splitting," *IEEE Trans. Parallel Distrib. Syst.*, vol. 35, no. 3, pp. 405–420, Mar. 2024.
- [7] Y. Liu et al., "Realizing corrupted-shard tolerance: A sharding blockchain with preserving global resilience," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2025, pp. 2099–2113.
- [8] M. Xu et al., "OPRE: Towards better availability of PCNs through recovering," *IEEE Trans. Dependable Secure Comput.*, vol. 22, no. 4, pp. 3839–3855, Jul. Aug. 2025.
- [9] X. Luo, K. Xue, Q. Sun, and J. Lu, "CrossChannel: Efficient and scalable cross-chain transactions through cross-and-off-blockchain micropayment channel," *IEEE Trans. Dependable Secure Comput.*, vol. 22, no. 1, pp. 649–663, Jan. Feb. 2025.
- [10] X. Qi et al., "Schain: Scalable concurrency over flexible permissioned blockchain," in *Proc. IEEE Int. Conf. Data Eng.*, 2023, pp. 1901–1913.
- [11] Z. Xu and L. Chen, "L2chain: Towards high-performance, confidential and secure layer-2 blockchain solution for decentralized applications," *Proc. VLDB Endowment*, vol. 16, no. 4, pp. 986–999, 2022.
- [12] H. Kalodner, S. Goldfeder, X. Chen, S. M. Weinberg, and E. W. Felten, "Arbitrum: Scalable, private smart contracts," in *Proc. USENIX Secur. Symp. USENIX Assoc.*, 2018, pp. 1353–1370.
- [13] C. Xu, C. Zhang, J. Xu, and J. Pei, "SlimChain: Scaling blockchain transactions through off-chain storage and parallel processing," *Proc. VLDB Endowment*, vol. 14, no. 11, pp. 2314–2326, 2021.
- [14] T. Frassetto et al., "POSE: Practical off-chain smart contract execution," in *Proc. Netw. Distrib. System Secur. Symp.*, 2023, pp. 1–18.
- [15] Q. Liu et al., "veffChain: Enabling freshness authentication of rich queries over blockchain databases," *IEEE Trans. Knowl. Data Eng.*, vol. 36, no. 5, pp. 2285–2300, May 2024.
- [16] H. Wang et al., "V2FS: A verifiable virtual filesystem for multi-chain query authentication," in *Proc. IEEE 40th Int. Conf. Data Eng.*, 2024, pp. 1999–2011.
- [17] H. Wang, C. Xu, C. Zhang, J. Xu, Z. Peng, and J. Pei, "vChain: Optimizing verifiable blockchain boolean range queries," in *Proc. IEEE Int. Conf. Data Eng.*, 2022, pp. 1927–1940.
- [18] X. Li, C. Weng, Y. Xu, X. Wang, and J. Rogers, "ZKSQ: Verifiable and efficient query evaluation with zero-knowledge proofs," *Proc. VLDB Endowment*, vol. 16, no. 8, pp. 1804–1816, 2023.
- [19] B. Gu, J. Fang, and F. Nawab, "PoneglyphDB: Efficient non-interactive zero-knowledge proofs for arbitrary SQL-query verification," *Proc. ACM Manage. Data*, vol. 3, no. 1, pp. 1–27, 2025.
- [20] Q. Liu et al., "MPV: Enabling fine-grained query authentication in hybrid-storage blockchain," *IEEE Trans. Knowl. Data Eng.*, vol. 36, no. 7, pp. 3297–3311, Jul. 2024.
- [21] D. Papadopoulos, S. Papadopoulos, and N. Triandopoulos, "Taking authenticated range queries to arbitrary dimensions," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2014, pp. 819–830.
- [22] Y. Zhang, J. Katz, and C. Papamantou, "IntegriDB: Verifiable SQL for outsourced databases," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2015, pp. 1480–1491.
- [23] Ethereum, "Optimistic rollups," 2025, Ethereum docs, Accessed: Dec., 2025. [Online]. Available: <https://ethereum.org/en/developers/docs/scaling/optimistic-rollups/>
- [24] P. Ruan et al., "Blockchains vs. distributed databases: Dichotomy and fusion," in *Proc. Int. Conf. Manage. Data.*, 2021, pp. 1504–1517.
- [25] Z. Ge, D. Loghin, B. C. Ooi, P. Ruan, and T. Wang, "Hybrid blockchain database systems: Design and performance," in *Proc. VLDB Endowment*, vol. 15, no. 5, 2022, pp. 1092–1104.

- [26] Y. Peng, M. Du, F. Li, R. Cheng, and D. Song, "FalconDB: Blockchain-based collaborative database," in *Proc. Int. Conf. Manage. Data.*, 2020, pp. 637–652.
- [27] C. Zhang, C. Xu, J. Xu, Y. Tang, and B. Choi, "GEM²-tree: A gas-efficient structure for authenticated range queries in blockchain," in *Proc. IEEE 35th Int. Conf. Data Eng.*, 2019, pp. 842–853.
- [28] C. Zhang, C. Xu, H. Wang, J. Xu, and B. Choi, "Authenticated keyword search in scalable hybrid-storage blockchains," in *Proc. IEEE Int. Conf. Data Eng.*, 2021, pp. 996–1007.
- [29] Y. Zhu, Z. Zhang, C. Jin, A. Zhou, and Y. Yan, "SEBDB: Semantics empowered blockchain database," in *Proc. IEEE Int. Conf. Data Eng.*, 2019, pp. 1820–1831.
- [30] C. Yue et al., "GlassDB: An efficient verifiable ledger database system through transparency," in *Proc. VLDB Endowment*, vol. 16, no. 6, 2023, pp. 1359–1371.
- [31] Hyperledger, "Hyperledger fabric tutorials: Using couchDB," 2022, Hyperledger Fabric Docs, Accessed: Dec., 2025. [Online]. Available: https://hyperledger-fabric.readthedocs.io/en/release-2.4/couchdb_tutorial.html
- [32] F. Li, M. Hadjieleftheriou, G. Kollios, and L. Reyzin, "Dynamic authenticated index structures for outsourced databases," in *Proc. ACM Int. Conf. Manage. Data (SIGMOD)*, 2006, pp. 121–132.
- [33] C. Papamanthou, R. Tamassia, and N. Triandopoulos, "Optimal verification of operations on dynamic sets," in *Proc. Annu. Cryptol. Conf. (CRYPTO)*, 2011, pp. 91–110.
- [34] R. Canetti, O. Paneth, D. Papadopoulos, and N. Triandopoulos, "Verifiable set operations over outsourced databases," in *Proc. Int. Workshop Public Key Cryptogr.*, 2014, pp. 113–130.
- [35] D. Leung, Y. Gilad, S. Gorbunov, N. Zeldovich, and L. Reyzin, "Aardvark: An asynchronous authenticated dictionary with applications to account-based cryptocurrencies," in *Proc. USENIX Secur. Symp. USENIX Assoc.*, 2022, pp. 4237–4254.
- [36] M. A. Al Sibahee et al., "Bilinear pairing and diffie-hellman based anonymous authentication protocol for the Internet of Vehicles," *IEEE Trans. Dependable Secure Comput.*, vol. 23, no. 2, pp. 2635–2647, Mar./Apr. 2026.
- [37] S. Gorbunov, L. Reyzin, H. Wee, and Z. Zhang, "Pointproofs: Aggregating proofs for multiple vector commitments," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2020, pp. 2007–2023.



Xinyi Luo received the BS degree in information security from School of the Gifted Young, University of Science and Technology of China (USTC), in 2020, and the PhD degree in information security with the School of Cyber Science and Technology, USTC, in 2025. Her research interests include Blockchain, Network security and Applied Cryptography.



Meiqi Li received the BS degree in information security from School of Cyber Science and Technology, University of Science and Technology of China (USTC), in 2021. She is currently working toward the PhD degree in information security with the School of Cyber Science and Technology, USTC. Her research interests include Blockchain, Network security and Applied Cryptography.



Kaiping Xue (Senior Member, IEEE) received the bachelor's degree from the Department of Information Security, University of Science and Technology of China (USTC), in 2003 and the doctor's degree from the Department of Electronic Engineering and Information Science, USTC, in 2007. From 2012 to 2013, he was a postdoctoral researcher with the Department of Electrical and Computer Engineering, University of Florida. He is currently a professor with the School of Cyber Science and Technology, USTC. He is also the Director of Network and Information Center, USTC. His research interests include future Internet architecture design, transmission optimization and network security. He is on the Editorial Board of several journals, including the *IEEE Transactions on Dependable and Secure Computing*, *IEEE Transactions on Wireless Communications*, and *IEEE Transactions on Network and Service Management*. He is an IET fellow.



Yunshu Wang received the bachelor's degree in cybersecurity from the School of Cyber Science and Engineering, Huazhong University of Science and Technology, in 2024. She is currently working toward the graduation degree with the School of Cyber Science and Technology, University of Science and Technology of China. Her research interests include blockchain and verifiable queries.



Yingjie Xue (Member, IEEE) received the bachelor's degree from the Department of Information Security, University of Science and Technology of China (USTC), in 2015, the master's degree from the Department of Electronic Engineering and Information Science, USTC, in 2018, and the PhD degree in computer science from Brown University, in 2023. She was an assistant professor with the Hong Kong University of Science and Technology (Guangzhou), Financial Technology Thrust, from 2023 to 2024. She is currently an associate researcher with the School of Cyber Science and Technology, USTC. Her research interests include privacy computing, blockchain and network security.



Lutong Chen (Member, IEEE) received the bachelor's and Ph.D degrees from the School of Cyber Science and Technology, University of Science and Technology of China in 2020 and 2024, respectively. He is currently an engineer with the Network and Information Center, USTC. His research interests include quantum networking and network security.